RMIT UNIVERSITY

# COSC2101/COSC2732 Software Engineering: Process and Tools
## Team Project and Final Demo

| | |
|---|---|
| **Assessment Type** | Group assignment |
| **Due Date** | • Milestone 1: Week 4, 20 November 2026, 10%<br>• Milestone 2: Week 7, 11 December 2026, 15%<br>• Milestone 3: Week 11, 8 January 2027, 15%<br>• Final Demo: Week 12, 11 January 2027 (submission due), presentation during tutorial session, 10% |
| **Marks** | 50 |
| **Version** | 1.0 |

## 1. Overview

You are required to develop a web application based on the assigned project scenario, using appropriate technologies, processes and tools, particularly those supporting collaborative project and code development.

### 1.1. Purpose of Assignment

You will simulate a software development team working with a client. You are expected to apply software engineering principles throughout the development process, from requirements gathering and system design to implementation, testing and delivery.

Your tutor will act as your client and provide feedback throughout the semester. Your team is responsible for identifying detailed requirements, proposing a suitable solution and making development decisions based on the assigned scenario.

### 1.2. Project Scenario

**Project Title:** RentXe - Online Motorbike and Car Rental Marketplace

**Project Description:** RentXe is a web application designed to connect vehicle rental companies with customers looking to rent motorbikes and cars. The platform aims to simplify the vehicle rental process by allowing customers to search for available vehicles, compare rental prices and make reservations online. The application will provide a centralised marketplace where multiple rental companies can register and manage their vehicle listings, rental prices and availability. Customers can browse vehicles from different rental companies, submit booking requests and manage their rentals. Rental companies can process booking requests, manage vehicle collections and returns, while administrators can oversee platform operations and manage registered users and companies.

**Key Features:** The application should support the following core feature areas.

1. **User and Profile Management:**
   - Customers can register, log in and manage their profiles.
   - Rental companies can register and manage their company profiles.
   - Administrators can manage user accounts and oversee registered rental companies.

2. **Vehicle Management:**
   - Rental companies can add, update and remove their motorbike and car listings.
   - Each listing should include basic vehicle information, such as vehicle type, brand, model, rental price and availability.

3. **Vehicle Search and Comparison:**
   - Customers can search for available motorbikes and cars.
   - The system should support basic filtering by vehicle type, rental price, brand and location.
   - Customers can view vehicle details and compare rental options from different companies.

4. **Rental Booking and Management:**
   - Customers can select rental dates and submit booking requests.
   - Rental companies can review, approve or reject booking requests.
   - Customers can view their booking status and cancel reservations where permitted.

5. **Rental and Return Management:**
   - Rental companies can record vehicle collection and return.
   - The system should support basic rental status tracking and vehicle condition records.
   - The system should calculate rental fees based on the selected rental period.

6. **Reviews and Ratings:**
   - Customers can submit ratings and reviews after completing a rental.
   - Customers can view feedback about vehicles and rental companies.

**Project Scope:** The six feature areas above define the core scope of the RentXe application. Teams are expected to analyse the scenario, identify detailed functional and non-functional requirements, and determine suitable workflows and business rules.

Teams should plan and develop the application incrementally throughout the project, following the milestone requirements and submission instructions provided on Canvas.

**Custom Features (Additional Opportunities for Higher Achievement):** Teams may propose and implement one or two additional features beyond the core requirements to demonstrate creativity and technical capability. The quality, complexity, relevance and successful implementation of these features will be considered in the assessment. The following are examples of possible enhancements. Teams may select from this list or propose their own ideas. All proposed custom features must be discussed with and approved by the tutor before implementation.

1. **Personalised Vehicle Recommendations:** Recommend vehicles based on customer preferences and previous bookings.
2. **AI Rental Assistant:** Provide an assistant to help customers find vehicles and answer questions about rental services.
3. **Vehicle Damage Detection:** Use image analysis to identify potential vehicle damage, such as scratches, dents and cracks, from photographs taken during vehicle collection and return.
4. **Review Sentiment Analysis:** Analyse customer reviews to identify common complaints, positive feedback and service quality trends.

**Deliverables:**

- A functional application that can be deployed or run in a documented environment.
- User documentation describing the application's features and providing guidance on how to use it.
- Testing and quality assurance reports documenting the testing activities, results and any identified defects or usability issues.
- A presentation and demonstration of the completed application and the processes and tools used to develop it.

The following are the overall project deliverables. Refer to the submission instructions for each milestone on Canvas for the specific artefacts and requirements. The project is organised into three milestones:

- Milestone 1: Requirements Gathering and System Design
- Milestone 2: MVP and Main Construction Sprints
- Milestone 3: Application Completion and Delivery

## 1.3. Group Work

This assignment is to be performed in groups of five.

- You will work as a Scrum team.
- Your group must be registered through the designated course process by the end of Week 2.
- Your group must assign Scrum responsibilities, including Scrum Master and Product Owner. All members are expected to be self-organised, self-managed and contribute throughout the project.
- Peer reviews and project activity records may be used to assess individual contributions and help resolve disputes.

If you experience problems working with your group, inform your tutor as soon as possible so that the issues can be addressed. Regular progress updates should help the team communicate its status.

All team members must contribute across the project, including documentation, implementation, integration and testing. Group meetings are compulsory. Use the following tools to manage collaboration:

- GitHub for source code sharing and version control.
- GitHub Projects for the Scrum board, product backlog and task management.
- Microsoft Teams for group discussion and communication.
- Classroom 50 for assignment distribution and submission where configured for the course.

## 1.4. Tools and Technologies

The following technology stack is prescribed for this project:

- Code repository: GitHub (each group will be assigned to their own private GitHub repository after forming in Week 3)
- Task management: GitHub Projects (default)
- Group communication: Microsoft Teams
- Programming language: Java (version 21 or later). All team members must use the same Java version.
- Frontend/UI: HTML, CSS and Javascript, or React (optional for teams wishing to build a richer frontend)
- Web framework: Spring Boot
- Build tool: Maven (latest stable version)

- CI/CD tool: GitHub Actions
- Recommended data store: H2 for development and testing, or MySQL for development, testing and deployment. Teams may use MySQL throughout the project if preferred.
- Unit testing framework: JUnit 5
- Containerization: Docker
- Documentation: Office 365 online

External code, libraries and services must be appropriately acknowledged, and their licences and usage conditions must be respected.

### 1.5. Progress

Agile software development requires teams to work collaboratively through regular meetings, short development sprints, frequent code commits and continuous integration. Teams must apply Scrum throughout the project and develop the application incrementally across the milestones.

Teams must commit frequently during each sprint and whenever code is added or changed. Marks may be deducted if the entire codebase is submitted in one large commit or push. The repository should demonstrate incremental development across multiple iterations.

All team members are expected to contribute to coding and understand the overall application. Pair programming and collaboration are encouraged.

Starting in Week 3, the weekly tutorial session will act as the team's Scrum stand-up meeting, with the tutor playing the role of client. The Scrum Master is responsible for ensuring that the meeting takes place, remains focused and stays on time.

Each team member should provide a brief update covering:

1. What I did last week (progress since the previous tutorial).
2. What I plan to do next (tasks or features for the coming week).
3. What is blocking me (issues or help needed).

Expectations:

- Every team member must contribute to the stand-up.
- Updates should be specific and concise - focus on progress and next actions.
- Be prepared to show evidence of your work (e.g., Git commits, pull requests, screenshots or working features).
- Regular attendance and participation contribute to teamwork and professionalism assessment.

## 2. Learning Outcomes

This assessment relates to the course learning outcomes:

1. Utilise software engineering processes and tools for best-practice development of software systems.
2. Identify, analyse, compare and contrast processes and tools used in different phases of the software engineering lifecycle.
3. Cooperate and contribute in a team environment, follow agreed team protocols, and manage conflict.
4. Run and document meetings.

5. Plan, identify and apply processes, standards and tools throughout a substantial software development project.
6. Clearly present and discuss the developed solution with a variety of audiences.

# 3. Deliverables

## 3.1 Mandatory Requirements

As part of your project, you must:

- Use Scrum as the project methodology and follow sprint activities.
- Conduct regular Scrum stand-ups (at least four per week, including one during class; other meetings may be online).
- Document meetings and include the required records in milestone submissions.
- Include refactoring activities throughout the project.
- Contribute to design, coding, testing, documentation, management and integration.
- Follow appropriate coding standards and maintainable design principles.
- Apply a coherent, loosely coupled architecture and the MVC pattern where appropriate.
- Use GitHub for source code management and proper version control.
- Use GitHub Projects to track tasks, responsibilities and work in progress. Keep the board up to date.
- Maintain a working application at the end of each sprint.

Based on the RentXe scenario, each team must identify the relevant stakeholders and define the functional and non-functional requirements, workflows and business rules for its proposed solution. These requirements should guide the design and implementation of the application.

## 3.2. Project Milestones and Final Demo

The team project is organised into three milestones and a final demonstration. Each milestone has its own requirements, submission instructions and assessment criteria.

Students should refer to the corresponding milestone pages on Canvas for detailed instructions, deliverables and submission requirements.

| Milestone | Title | Due date | Weight |
|---|---|---|---|
| Milestone 1 | Requirements Gathering and System Design | 20 Nov 2026 | 10% |
| Milestone 2 | MVP and Main Construction Sprints | 11 Dec 2026 | 15% |
| Milestone 3 | Application Completion and Delivery | 8 Jan 2027 | 15% |
| Final Demo | Project Demonstration | 11 Jan 2027 | 10% |

At the end of the project, your group will present and demonstrate the application and discuss the process and tools used to develop it. Your presentation should:

- Demonstrate the implemented core functionality and enhancements.
- Explain the project process, sprint retrospectives, learning outcomes and project progress.
- Discuss the tools used and how they supported development.
- Present relevant testing results and quality assurance activities.
- Explain deployment and containerisation.

The purpose of the presentation is to demonstrate the quality of the software and the team's overall work. Each team member must be prepared to explain and show their own contribution.

## 4. Submission

Detailed submission instructions and required artefacts for each milestone are provided on Canvas. Submit all required materials through the designated submission process. Your GitHub repository and GitHub Projects board should provide clear evidence of project progress and individual contributions.

## 5. Academic integrity and plagiarism (standard warning)

Academic integrity requires the honest presentation of your work and appropriate acknowledgement of the work of others. Take care to acknowledge words, data, diagrams, models, code, frameworks and ideas that you quote, summarise, paraphrase or discuss, and provide references that allow the sources to be located.

External source code, APIs and libraries must be appropriately acknowledged and used in accordance with their licences. You are responsible for understanding and being able to explain the code and other artefacts submitted by your team. Any use of generative AI must comply with university policy and the course instructions.

Failure to acknowledge sources may constitute plagiarism or academic misconduct. For further information, refer to the RMIT University academic integrity policies:
https://www.rmit.edu.au/students/student-essentials/rights-and-responsibilities/academic-integrity